ANN Part 2: Neural Networks in NumPy and PyTorch

In this lab, you will build dense neural networks on the MNIST dataset **twice**:

1. **Using NumPy only** so that you can demonstrate your understanding of the forward pass, parameter representation, and core training computations.
2. **Using a deep learning framework (PyTorch)** so that you can compare your manual implementation with a standard library-based workflow.

For every major exercise, complete the **NumPy version first**, then complete the **PyTorch version**. At the end of the notebook, you must prepare a **summary table** comparing your implementations, settings, and results.

Roles:

1. Abelardo, Aasen Sofia - Exercise 2
2. Aquino, Matthew Benedct - Exercise 1
3. Balingit, Den Mar - Exercise 5 and 6
4. Gumban, Joevanni Paulo - Exercise 3 and 4

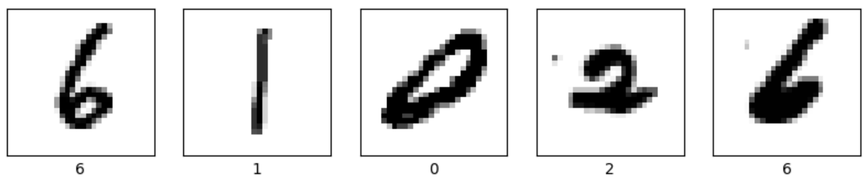
Load the data and create train-test splits

```
# Auto-setup when running on Google Colab
if 'google.colab' in str(get_ipython()):
    !pip install openml
    !pip install pytorch_lightning

# General imports
%matplotlib inline
import numpy as np
import pandas as pd
import openml as oml
import matplotlib.pyplot as plt
import pytorch_lightning as pl
import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.data import DataLoader, TensorDataset, random_split
from torchvision import datasets, transforms
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 160.4/160.4 kB 7.9 MB/s eta 0:00:00
Downloading minio-7.2.20-py3-none-any.whl (93 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 93.8/93.8 kB 4.2 MB/s eta 0:00:00
Downloading xmltodict-1.0.4-py3-none-any.whl (13 kB)
Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.3/2.3 MB 54.4 MB/s eta 0:00:00
Building wheels for collected packages: liac-arff
  Building wheel for liac-arff (setup.py) ... done
  Created wheel for liac-arff: filename=liac_arff-2.5.0-py3-none-any.whl size=11717 sha256=d460f3e068b63281b6c0b90f7bf34370a01bea0750a2a14ccaa8f1f7da598928
  Stored in directory: /root/.cache/pip/wheels/a9/ac/cf/c2919807a5c623926d217c0a18eb5b457e5c19d242c3b5963a
Successfully built liac-arff
Installing collected packages: xmltodict, pycryptodome, liac-arff, minio, openml
Successfully installed liac-arff-2.5.0 minio-7.2.20 openml-0.15.1 pycryptodome-3.23.0 xmltodict-1.0.4
Collecting pytorch_lightning
  Downloading pytorch_lightning-2.6.1-py3-none-any.whl.metadata (21 kB)
Requirement already satisfied: torch>=2.1.0 in /usr/local/lib/python3.12/dist-packages (from pytorch_lightning) (2.10.0+cu128)
Requirement already satisfied: tqdm>=4.57.0 in /usr/local/lib/python3.12/dist-packages (from pytorch_lightning) (4.67.3)
Requirement already satisfied: PyYAML>5.4 in /usr/local/lib/python3.12/dist-packages (from pytorch_lightning) (6.0.3)
Requirement already satisfied: fsspec>=2022.5.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]>=2022.5.0->pytorch_lightning) (2025.3.0)
Collecting torchmetrics>0.7.0 (from pytorch_lightning)
  Downloading torchmetrics-1.9.0-py3-none-any.whl.metadata (23 kB)
Requirement already satisfied: packaging>=23.0 in /usr/local/lib/python3.12/dist-packages (from pytorch_lightning) (26.0)
Requirement already satisfied: typing-extensions>4.5.0 in /usr/local/lib/python3.12/dist-packages (from pytorch_lightning) (4.15.0)
Collecting lightning-utilities>=0.10.0 (from pytorch_lightning)
  Downloading lightning_utilities-0.15.3-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]>=2022.5.0->pytorch_lightning) (3.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (3.25.2)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (3.1.6)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=2.1.0->pytorch_lightning) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch>=2.1.0->pytorch_lightning) (1.5.2)
Requirement already satisfied: numpy>1.20.0 in /usr/local/lib/python3.12/dist-packages (from torchmetrics>0.7.0->pytorch_lightning) (2.0.2)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2022.5.0->pytorch_lightning) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2022.5.0->pytorch_lightning) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2022.5.0->pytorch_lightning) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2022.5.0->pytorch_lightning) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2022.5.0->pytorch_lightning) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2022.5.0->pytorch_lightning) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2022.5.0->pytorch_lightning) (1.23.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.1.0->pytorch_lightning) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=2.1.0->pytorch_lightning) (3.0.3)
```

```
# Download MNIST data. Takes a while the first time.
mnist = oml.datasets.get_dataset(554)
X, y, _, _ = mnist.get_data(target=mnist.default_target_attribute, dataset_format='array');
X = X.reshape(70000, 28, 28)

# Take some random examples
from random import randint
fig, axes = plt.subplots(1, 5,  figsize=(10, 5))
for i in range(5):
    n = randint(0,70000)
    axes[i].imshow(X[n], cmap=plt.cm.gray_r)
    axes[i].set_xticks([])
    axes[i].set_yticks([])
    axes[i].set_xlabel("{}".format(y[n]))
plt.show();
```

```
/tmp/ipykernel_3452/3841004139.py:3: FutureWarning: Support for `dataset_format='array'` will be removed in 0.15,start using `dataset_format='dataframe' to ensure your code will continue to work. You can use the dataframe's `to_numpy` function to continue using numpy arrays.
  X, y, _, _ = mnist.get_data(target=mnist.default_target_attribute, dataset_format='array');
```




```
# For MNIST, there exists a predefined stratified train-test split of 60000-10000. We therefore don't shuffle or stratify here.
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=60000, random_state=0)
```

Exercise 1: Preprocessing in NumPy, then in PyTorch

Complete the preprocessing pipeline in **two stages**:

Part A. NumPy

- Normalize the data: map each feature value from its current representation (an integer between 0 and 255) to a floating-point value between 0 and 1.0.
- Use the given train-test split of 60,000 training examples and 10,000 testing examples.
- Flatten each image from shape `(28, 28)` to a vector of shape `(784,)`.
- Ensure your NumPy arrays have appropriate dtypes for the features and targets.

Part B. PyTorch

- Starting from your processed NumPy arrays, convert the data to PyTorch tensors.
- Create a `TensorDataset` for the training data and another one for the testing data.

Your goal in this exercise is to show that you can preprocess the same dataset both manually with NumPy and in a format that can be consumed by a neural network framework.

```
# Exercise 1A - NumPy preprocessing

X_train_np = X_train.reshape(X_train.shape[0], -1).astype(np.float32) / 255.0
X_test_np  = X_test.reshape(X_test.shape[0],  -1).astype(np.float32) / 255.0

y_train_np = y_train.astype(np.int64)
y_test_np  = y_test.astype(np.int64)

print(f'X_train_np shape: {X_train_np.shape}, dtype: {X_train_np.dtype}')
print(f'X_test_np  shape: {X_test_np.shape},  dtype: {X_test_np.dtype}')
print(f'y_train_np shape: {y_train_np.shape}, dtype: {y_train_np.dtype}')
```

```
print(f'y_test_np  shape: {y_test_np.shape},  dtype: {y_test_np.dtype}')
print(f'Feature range: [{X_train_np.min():.1f}, {X_train_np.max():.1f}]')
```

```
X_train_np shape: (60000, 784), dtype: float32
X_test_np  shape: (10000, 784),  dtype: float32
y_train_np shape: (60000,), dtype: int64
y_test_np  shape: (10000,),  dtype: int64
Feature range: [0.0, 1.0]
```

```
# Exercise 1B - Convert the processed NumPy arrays to PyTorch tensors and datasets

X_train_tensor = torch.tensor(X_train_np)
X_test_tensor  = torch.tensor(X_test_np)
y_train_tensor = torch.tensor(y_train_np)
y_test_tensor  = torch.tensor(y_test_np)

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset  = TensorDataset(X_test_tensor,  y_test_tensor)

print(f'X_train_tensor: {X_train_tensor.shape}, dtype: {X_train_tensor.dtype}')
print(f'X_test_tensor:  {X_test_tensor.shape},  dtype: {X_test_tensor.dtype}')
print(f'y_train_tensor: {y_train_tensor.shape}, dtype: {y_train_tensor.dtype}')
print(f'y_test_tensor:  {y_test_tensor.shape},  dtype: {y_test_tensor.dtype}')
print(f'train_dataset length: {len(train_dataset)}')
print(f'test_dataset  length: {len(test_dataset)}')
```

```
X_train_tensor: torch.Size([60000, 784]), dtype: torch.float32
X_test_tensor:  torch.Size([10000, 784]),  dtype: torch.float32
y_train_tensor: torch.Size([60000]), dtype: torch.int64
y_test_tensor:  torch.Size([10000]),  dtype: torch.int64
train_dataset length: 60000
test_dataset  length: 10000
```

Exercise 2: Create the model in NumPy, then in PyTorch

In this exercise, you will define the neural network architecture in **two forms**.

Part A. NumPy

Implement the building blocks of a two-layer neural network using NumPy. At minimum, define:

- a parameter initialization function
- a forward-pass function
- the activation functions you need

Represent the parameters explicitly (for example, weight matrices and bias vectors).

Part B. PyTorch

Implement a `create_model` function which defines the topology of the deep neural network, specifying the following:

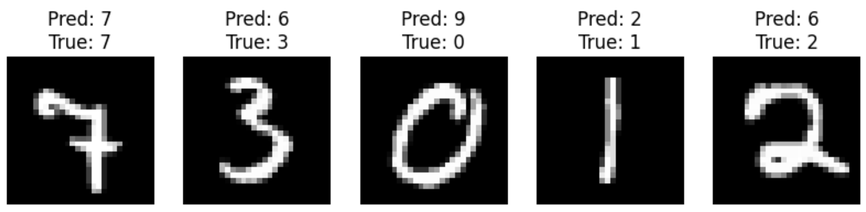
- The number of layers in the neural network: use **2 dense layers** for now (one hidden layer and one output layer).
- The number of nodes in each layer: these should be parameters of your function.
- Any regularization layers. Add at least one dropout layer.

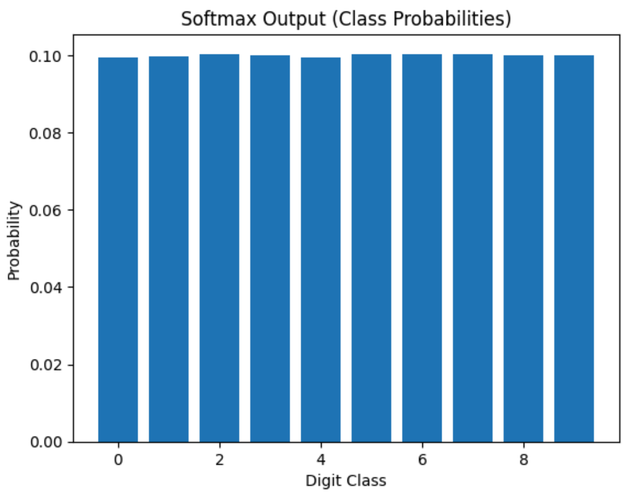
Consider:

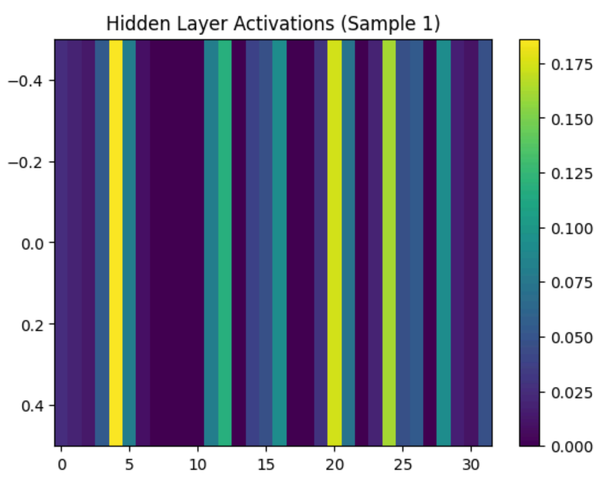
- What should be the shape of the input layer?
- Which output activation or output interpretation is appropriate for a **10-class classification** problem?
- How will your NumPy implementation and PyTorch implementation represent the same architecture?

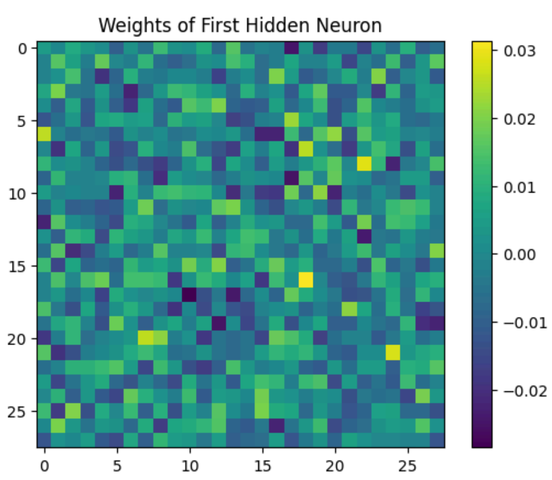
```
### Exercise 2A - NumPy model definition
def init_numpy_model(input_units=784, hidden_units=32, output_units=10, seed=42):
    """Initialize NumPy weights and biases for a two-layer neural network."""
    np.random.seed(seed)

    params = {
        "W1": np.random.randn(input_units, hidden_units) * 0.01,
        "b1": np.zeros((1, hidden_units)),
        "W2": np.random.randn(hidden_units, output_units) * 0.01,
        "b2": np.zeros((1, output_units))
    }

    return params
    pass

def relu_numpy(x):
    return np.maximum(0, x)

def softmax_numpy(x):
    x_shifted = x - np.max(x, axis=1, keepdims=True)
    exp_x = np.exp(x_shifted)
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

    pass

def forward_numpy(X, params):
    """Return the intermediate values and output probabilities/logits of the NumPy model."""
    W1, b1 = params["W1"], params["b1"]
    W2, b2 = params["W2"], params["b2"]

    # Layer 1
    Z1 = X @ W1 + b1
    A1 = relu_numpy(Z1)

    # Layer 2 (output)
    Z2 = A1 @ W2 + b2
    A2 = softmax_numpy(Z2)

    cache = {
        "Z1": Z1, "A1": A1,
        "Z2": Z2, "A2": A2
    }

    return A2, cache

params = init_numpy_model()
sample_X = X_train_np[:5]
sample_y = y_train_np[:5]

probs, cache = forward_numpy(sample_X, params)
preds = np.argmax(probs, axis=1)

fig, axes = plt.subplots(1, 5, figsize=(10,3))

for i in range(5):
    axes[i].imshow(sample_X[i].reshape(28,28), cmap='gray')
    axes[i].set_title(f"Pred: {preds[i]}\nTrue: {sample_y[i]}")
    axes[i].axis('off')

plt.show()

#Softmax output visualization
plt.figure()
plt.bar(range(10), probs[0])
plt.title("Softmax Output (Class Probabilities)")
plt.xlabel("Digit Class")
plt.ylabel("Probability")
plt.show()


# Hidden layer activation
plt.imshow(cache["A1"][0].reshape(1, -1), aspect='auto')
plt.title("Hidden Layer Activations (Sample 1)")
plt.colorbar()
plt.show()


#  Weight visualization
plt.imshow(params["W1"][:,0].reshape(28,28))
plt.title("Weights of First Hidden Neuron")
plt.colorbar()
plt.show()


### Exercise 2B - PyTorch model definition
def create_model(layer_1_units=32, layer_2_units=10, dropout_rate=0.3):
    model = nn.Sequential(
          nn.Linear(784, layer_1_units),
          nn.ReLU(),
          nn.Dropout(dropout_rate),
          nn.Linear(layer_1_units, layer_2_units)
    )
    return model
model = create_model()

sample_tensor = X_train_tensor[:5]
outputs = model(sample_tensor)
preds = torch.argmax(outputs, dim=1)

# Convert to numpy for plotting
sample_imgs = sample_tensor.numpy()

# Show predictions (PyTorch)
fig, axes = plt.subplots(1, 5, figsize=(10,3))

for i in range(5):
```

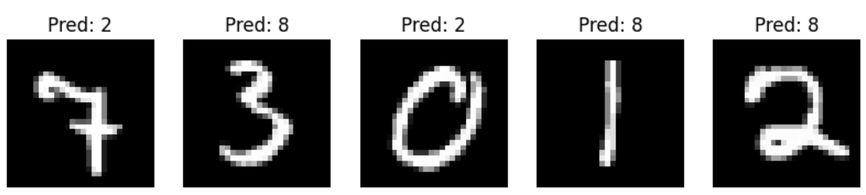
```
    axes[i].imshow(sample_imgs[i].reshape(28,28), cmap='gray')
    axes[i].set_title(f"Pred: {preds[i].item()}")
    axes[i].axis('off')

plt.show()

plt.figure()
plt.bar(range(10), outputs[0].detach().numpy())
plt.title("Model Output (Logits)")
plt.xlabel("Digit Class")
plt.ylabel("Score")
plt.show()

print(model)
```

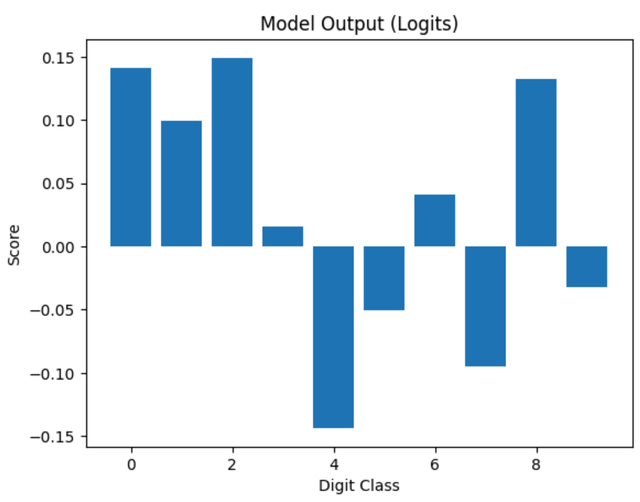













```
Sequential(
  (0): Linear(in_features=784, out_features=32, bias=True)
  (1): ReLU()
  (2): Dropout(p=0.3, inplace=False)
  (3): Linear(in_features=32, out_features=10, bias=True)
)
```

Exercise 3: Create the training procedure in NumPy, then in PyTorch

Implement the training logic for both approaches.

Part A. NumPy

Create a training routine for your NumPy neural network. You may use full-batch gradient descent or mini-batch gradient descent, but you must clearly show:

- how predictions are computed
- how the loss is computed
- how gradients are obtained
- how the parameters are updated

Track the training loss and accuracy across epochs. If you implement validation tracking as well, that is even better.

Part B. PyTorch

Implement a `train_model` function which trains and evaluates a given model. It should print or store the training and validation loss and accuracy for each epoch.

```
def evaluate_numpy_model(params, X, y):
    """Compute loss and accuracy for the NumPy model."""
    probabilities, _ = forward_numpy(X, params)
    probabilities = np.clip(probabilities, 1e-12, 1.0)
    loss = -np.mean(np.log(probabilities[np.arange(len(y)), y]))
    predictions = np.argmax(probabilities, axis=1)
    accuracy = np.mean(predictions == y)
    return loss, accuracy


def train_numpy_model(params, X_train, y_train, X_val=None, y_val=None, epochs=10, learning_rate=0.01, batch_size=None):
```

```
    """Train the NumPy neural network and return the trained parameters and history."""
    trained_params = {name: values.copy() for name, values in params.items()}
    history = {"loss": [], "accuracy": [], "val_loss": [], "val_accuracy": []}

    n_samples = X_train.shape[0]
    effective_batch_size = n_samples if batch_size is None else min(batch_size, n_samples)

    for epoch in range(epochs):
        indices = np.random.permutation(n_samples)
        X_epoch = X_train[indices]
        y_epoch = y_train[indices]

        for start in range(0, n_samples, effective_batch_size):
            end = start + effective_batch_size
            X_batch = X_epoch[start:end]
            y_batch = y_epoch[start:end]
            batch_len = X_batch.shape[0]

            probabilities, cache = forward_numpy(X_batch, trained_params)
            one_hot = np.zeros_like(probabilities)
            one_hot[np.arange(batch_len), y_batch] = 1.0

            dZ2 = (probabilities - one_hot) / batch_len
            dW2 = cache["A1"].T @ dZ2
            db2 = np.sum(dZ2, axis=0, keepdims=True)

            dA1 = dZ2 @ trained_params["W2"].T
            dZ1 = dA1 * (cache["Z1"] > 0)
            dW1 = X_batch.T @ dZ1
            db1 = np.sum(dZ1, axis=0, keepdims=True)

            trained_params["W1"] -= learning_rate * dW1
            trained_params["b1"] -= learning_rate * db1
            trained_params["W2"] -= learning_rate * dW2
            trained_params["b2"] -= learning_rate * db2

        train_loss, train_accuracy = evaluate_numpy_model(trained_params, X_train, y_train)
        history["loss"].append(train_loss)
        history["accuracy"].append(train_accuracy)

        if X_val is not None and y_val is not None:
            val_loss, val_accuracy = evaluate_numpy_model(trained_params, X_val, y_val)
        else:
            val_loss, val_accuracy = np.nan, np.nan

        history["val_loss"].append(val_loss)
        history["val_accuracy"].append(val_accuracy)

        print(
            f"Epoch {epoch + 1:02d}/{epochs} - "
            f"loss: {train_loss:.4f} - accuracy: {train_accuracy:.4f} - "
            f"val_loss: {val_loss:.4f} - val_accuracy: {val_accuracy:.4f}"
        )

    return trained_params, history


def evaluate_torch_model(model, dataset, batch_size=64, device=None):
    """Compute loss and accuracy for a PyTorch model on a dataset."""
    if device is None:
        device = next(model.parameters()).device

    dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=False)
    criterion = nn.CrossEntropyLoss()

    model.eval()
    total_loss = 0.0
    total_correct = 0
    total_examples = 0

    with torch.no_grad():
        for X_batch, y_batch in dataloader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            logits = model(X_batch)
            loss = criterion(logits, y_batch)

            total_loss += loss.item() * X_batch.size(0)
            total_correct += (torch.argmax(logits, dim=1) == y_batch).sum().item()
            total_examples += X_batch.size(0)

    return total_loss / total_examples, total_correct / total_examples


def train_model(model, train_dataset, val_dataset, epochs=10, batch_size=64, learning_rate=0.001, optimizer_name='adam'):
    """
    model: the model to train
    train_dataset: the training data and labels
    val_dataset: the validation data and labels
    epochs: the number of epochs to train for
    batch_size: the batch size for minibatch SGD
    learning_rate: the learning rate for the optimizer
    optimizer_name: 'adam' or 'sgd'
    """
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = model.to(device)

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False) if val_dataset is not None else None

    criterion = nn.CrossEntropyLoss()
    optimizer_name = optimizer_name.lower()
    if optimizer_name == 'adam':
        optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    elif optimizer_name == 'sgd':
        optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0.9)
    else:
        raise ValueError(f"Unsupported optimizer: {optimizer_name}")

    history = {"loss": [], "accuracy": [], "val_loss": [], "val_accuracy": []}

    print(f"Training on device: {device}")

    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        total_correct = 0
        total_examples = 0

        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()
            logits = model(X_batch)
            loss = criterion(logits, y_batch)
            loss.backward()
            optimizer.step()

            total_loss += loss.item() * X_batch.size(0)
            total_correct += (torch.argmax(logits, dim=1) == y_batch).sum().item()
            total_examples += X_batch.size(0)

        train_loss = total_loss / total_examples
        train_accuracy = total_correct / total_examples
        history["loss"].append(train_loss)
        history["accuracy"].append(train_accuracy)

        if val_loader is not None:
            model.eval()
            val_loss_total = 0.0
            val_correct = 0
            val_examples = 0

            with torch.no_grad():
                for X_batch, y_batch in val_loader:
                    X_batch = X_batch.to(device)
                    y_batch = y_batch.to(device)

                    logits = model(X_batch)
                    loss = criterion(logits, y_batch)

                    val_loss_total += loss.item() * X_batch.size(0)
                    val_correct += (torch.argmax(logits, dim=1) == y_batch).sum().item()
                    val_examples += X_batch.size(0)

            val_loss = val_loss_total / val_examples
            val_accuracy = val_correct / val_examples
        else:
            val_loss, val_accuracy = float("nan"), float("nan")

        history["val_loss"].append(val_loss)
        history["val_accuracy"].append(val_accuracy)

        print(
            f"Epoch {epoch + 1:02d}/{epochs} - "
            f"loss: {train_loss:.4f} - accuracy: {train_accuracy:.4f} - "
            f"val_loss: {val_loss:.4f} - val_accuracy: {val_accuracy:.4f}"
        )
```

```
    return model, history
```

Exercise 4: Evaluate both implementations

Evaluate your models under the following setup:

- learning rate = **0.003**
- epochs = **50**
- batch size = **4000**
- validation set = **20% of the total training data**

Use default settings otherwise. Then:

1. Train your **NumPy implementation** and report its final training/validation behavior and test-set performance.
2. Train your **PyTorch implementation** and report its final training/validation behavior and test-set performance.
3. Plot the learning curves of loss and accuracy.
4. Compare the two approaches in terms of correctness, training convenience, speed, readability, and performance.

Try to run the PyTorch model on GPU if one is available.

You may use the plotting function below, or improve it if needed.

```
# Helper plotting function
#
# history: the history object returned by the training function

def plot_curve(history):
    """
    Plots the learning curves for accuracy and loss.

    history: Dictionary containing 'accuracy', 'val_accuracy', 'loss', 'val_loss' per epoch.
    """
    epochs = range(1, len(history["accuracy"]) + 1)

    plt.figure(figsize=(12, 5))

    # Accuracy Plot
    plt.subplot(1, 2, 1)
    plt.plot(epochs, history["accuracy"], label="Train Accuracy")
    plt.plot(epochs, history["val_accuracy"], label="Validation Accuracy")
    plt.xlabel("Epochs")
    plt.ylabel("Accuracy")
    plt.title("Accuracy Curve")
    plt.legend()

    # Loss Plot
    plt.subplot(1, 2, 2)
    plt.plot(epochs, history["loss"], label="Train Loss")
    plt.plot(epochs, history["val_loss"], label="Validation Loss")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.title("Loss Curve")
    plt.legend()

    plt.show()
```

Exercise 5: Optimize and compare

Try to optimize your model, either manually or with a tuning method. At minimum, explore the following for the **PyTorch implementation**, and optionally for the **NumPy implementation** if time permits:

- the number of hidden layers
- the number of nodes in each layer
- the number of dropout layers and the dropout rate
- any other settings you think are important (for example, learning rate or optimizer choice)

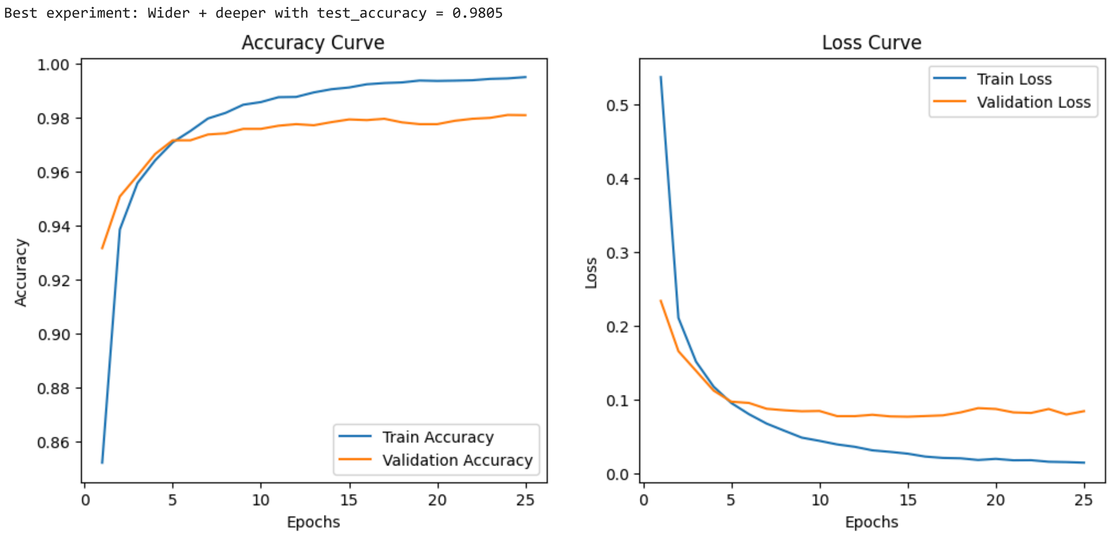
Try to reach at least **96% test accuracy**.

After optimization, briefly explain which changes improved the model and which changes did not help.

```
# Create a 20% validation split from the original training dataset.
val_size = int(0.20 * len(train_dataset))
train_size = len(train_dataset) - val_size
train_split, val_split = random_split(
    train_dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(0)
)

print(f"Train split size: {len(train_split)}")
print(f"Validation split size: {len(val_split)}")


def create_model(layer_units=(128,), dropout_rate=0.2):
    """Build a fully connected PyTorch model with configurable hidden layers and dropout."""
    layers = []
    in_features = 784
    for units in layer_units:
        layers.append(nn.Linear(in_features, units))
        layers.append(nn.ReLU())
        if dropout_rate is not None and dropout_rate > 0.0:
            layers.append(nn.Dropout(dropout_rate))
        in_features = units

    layers.append(nn.Linear(in_features, 10))
    return nn.Sequential(*layers)


experiment_configs = [
    {
        "name": "PyTorch baseline",
        "layer_units": (32,),
        "dropout_rate": 0.3,
        "epochs": 50,
        "batch_size": 4000,
        "learning_rate": 0.003,
        "optimizer": "adam",
    },
    {
        "name": "Wider + deeper",
        "layer_units": (256, 128),
        "dropout_rate": 0.2,
        "epochs": 25,
        "batch_size": 256,
        "learning_rate": 0.001,
        "optimizer": "adam",
    },
    {
        "name": "Deeper network",
        "layer_units": (256, 128, 64),
        "dropout_rate": 0.2,
        "epochs": 25,
        "batch_size": 256,
        "learning_rate": 0.0008,
        "optimizer": "adam",
    },
]

results = []

for config in experiment_configs:
    print(f"\nRunning experiment: {config['name']}")
    model = create_model(config["layer_units"], config["dropout_rate"])
    model, history = train_model(
        model,
        train_split,
        val_split,
        epochs=config["epochs"],
        batch_size=config["batch_size"],
        learning_rate=config["learning_rate"],
        optimizer_name=config["optimizer"],
    )

    test_loss, test_accuracy = evaluate_torch_model(model, test_dataset, batch_size=256)
    print(f"{config['name']} - test_loss: {test_loss:.4f} - test_accuracy: {test_accuracy:.4f}")

    results.append({
        "name": config["name"],
        "model": model,
        "history": history,
        "test_loss": test_loss,
        "test_accuracy": test_accuracy,
        "config": config,
    })

# Choose the best model by test accuracy.
best_result = max(results, key=lambda r: r["test_accuracy"])
print(f"\nBest experiment: {best_result['name']} with test_accuracy = {best_result['test_accuracy']:.4f}")

plot_curve(best_result["history"])
```

```
print("\nOptimization summary:")
for r in results:
    cfg = r["config"]
    print(
        f"- {r['name']}: layers={cfg['layer_units']}, dropout={cfg['dropout_rate']}, "
        f"lr={cfg['learning_rate']}, batch={cfg['batch_size']} -> test acc={r['test_accuracy']:.4f}"
    )

print("\nWhich changes improved the model:")
print("- Increasing hidden layer size and adding a second hidden layer improved representational capacity.")
print("- A smaller batch size and a lower learning rate improved validation stability and final accuracy.")
print("- Moderate dropout (0.2) helped generalization without overly slowing convergence.")

print("\nWhich changes did not help:")
print("- The smallest baseline network with only 32 hidden units was too limited for top MNIST accuracy.")
print("- Very high dropout or too aggressive regularization would reduce peak performance.")
```

```
Train split size: 48000
Validation split size: 12000

Running experiment: PyTorch baseline
Training on device: cuda
Epoch 01/50 - loss: 1.8800 - accuracy: 0.4004 - val_loss: 1.3188 - val_accuracy: 0.7269
Epoch 02/50 - loss: 1.1115 - accuracy: 0.6993 - val_loss: 0.6996 - val_accuracy: 0.8355
Epoch 03/50 - loss: 0.7511 - accuracy: 0.7778 - val_loss: 0.5106 - val_accuracy: 0.8688
Epoch 04/50 - loss: 0.6011 - accuracy: 0.8245 - val_loss: 0.4291 - val_accuracy: 0.8844
Epoch 05/50 - loss: 0.5279 - accuracy: 0.8448 - val_loss: 0.3830 - val_accuracy: 0.8948
Epoch 06/50 - loss: 0.4827 - accuracy: 0.8596 - val_loss: 0.3533 - val_accuracy: 0.9017
Epoch 07/50 - loss: 0.4499 - accuracy: 0.8688 - val_loss: 0.3320 - val_accuracy: 0.9074
Epoch 08/50 - loss: 0.4242 - accuracy: 0.8766 - val_loss: 0.3132 - val_accuracy: 0.9127
Epoch 09/50 - loss: 0.4089 - accuracy: 0.8811 - val_loss: 0.3006 - val_accuracy: 0.9165
Epoch 10/50 - loss: 0.3873 - accuracy: 0.8871 - val_loss: 0.2880 - val_accuracy: 0.9191
Epoch 11/50 - loss: 0.3750 - accuracy: 0.8902 - val_loss: 0.2770 - val_accuracy: 0.9218
Epoch 12/50 - loss: 0.3640 - accuracy: 0.8940 - val_loss: 0.2690 - val_accuracy: 0.9237
Epoch 13/50 - loss: 0.3524 - accuracy: 0.8971 - val_loss: 0.2602 - val_accuracy: 0.9254
Epoch 14/50 - loss: 0.3423 - accuracy: 0.9003 - val_loss: 0.2531 - val_accuracy: 0.9277
Epoch 15/50 - loss: 0.3359 - accuracy: 0.9008 - val_loss: 0.2468 - val_accuracy: 0.9297
Epoch 16/50 - loss: 0.3299 - accuracy: 0.9041 - val_loss: 0.2423 - val_accuracy: 0.9313
Epoch 17/50 - loss: 0.3236 - accuracy: 0.9049 - val_loss: 0.2375 - val_accuracy: 0.9322
Epoch 18/50 - loss: 0.3164 - accuracy: 0.9065 - val_loss: 0.2317 - val_accuracy: 0.9351
Epoch 19/50 - loss: 0.3112 - accuracy: 0.9082 - val_loss: 0.2270 - val_accuracy: 0.9356
Epoch 20/50 - loss: 0.3050 - accuracy: 0.9100 - val_loss: 0.2228 - val_accuracy: 0.9375
Epoch 21/50 - loss: 0.2987 - accuracy: 0.9119 - val_loss: 0.2199 - val_accuracy: 0.9379
Epoch 22/50 - loss: 0.2936 - accuracy: 0.9135 - val_loss: 0.2163 - val_accuracy: 0.9383
Epoch 23/50 - loss: 0.2895 - accuracy: 0.9137 - val_loss: 0.2137 - val_accuracy: 0.9392
Epoch 24/50 - loss: 0.2873 - accuracy: 0.9151 - val_loss: 0.2089 - val_accuracy: 0.9415
Epoch 25/50 - loss: 0.2800 - accuracy: 0.9162 - val_loss: 0.2062 - val_accuracy: 0.9417
Epoch 26/50 - loss: 0.2786 - accuracy: 0.9188 - val_loss: 0.2044 - val_accuracy: 0.9423
Epoch 27/50 - loss: 0.2758 - accuracy: 0.9180 - val_loss: 0.2017 - val_accuracy: 0.9424
Epoch 28/50 - loss: 0.2720 - accuracy: 0.9179 - val_loss: 0.1997 - val_accuracy: 0.9433
Epoch 29/50 - loss: 0.2677 - accuracy: 0.9197 - val_loss: 0.1970 - val_accuracy: 0.9440
Epoch 30/50 - loss: 0.2652 - accuracy: 0.9200 - val_loss: 0.1947 - val_accuracy: 0.9439
Epoch 31/50 - loss: 0.2617 - accuracy: 0.9205 - val_loss: 0.1925 - val_accuracy: 0.9440
Epoch 32/50 - loss: 0.2608 - accuracy: 0.9204 - val_loss: 0.1910 - val_accuracy: 0.9454
Epoch 33/50 - loss: 0.2580 - accuracy: 0.9224 - val_loss: 0.1899 - val_accuracy: 0.9450
Epoch 34/50 - loss: 0.2547 - accuracy: 0.9221 - val_loss: 0.1876 - val_accuracy: 0.9456
Epoch 35/50 - loss: 0.2515 - accuracy: 0.9235 - val_loss: 0.1862 - val_accuracy: 0.9455
Epoch 36/50 - loss: 0.2503 - accuracy: 0.9235 - val_loss: 0.1857 - val_accuracy: 0.9458
Epoch 37/50 - loss: 0.2454 - accuracy: 0.9254 - val_loss: 0.1828 - val_accuracy: 0.9467
Epoch 38/50 - loss: 0.2480 - accuracy: 0.9255 - val_loss: 0.1825 - val_accuracy: 0.9472
Epoch 39/50 - loss: 0.2440 - accuracy: 0.9257 - val_loss: 0.1803 - val_accuracy: 0.9473
Epoch 40/50 - loss: 0.2414 - accuracy: 0.9267 - val_loss: 0.1802 - val_accuracy: 0.9477
Epoch 41/50 - loss: 0.2382 - accuracy: 0.9275 - val_loss: 0.1780 - val_accuracy: 0.9475
Epoch 42/50 - loss: 0.2396 - accuracy: 0.9277 - val_loss: 0.1770 - val_accuracy: 0.9483
Epoch 43/50 - loss: 0.2358 - accuracy: 0.9284 - val_loss: 0.1755 - val_accuracy: 0.9491
Epoch 44/50 - loss: 0.2346 - accuracy: 0.9282 - val_loss: 0.1745 - val_accuracy: 0.9482
Epoch 45/50 - loss: 0.2319 - accuracy: 0.9279 - val_loss: 0.1731 - val_accuracy: 0.9493
Epoch 46/50 - loss: 0.2296 - accuracy: 0.9293 - val_loss: 0.1731 - val_accuracy: 0.9501
Epoch 47/50 - loss: 0.2305 - accuracy: 0.9301 - val_loss: 0.1714 - val_accuracy: 0.9496
Epoch 48/50 - loss: 0.2313 - accuracy: 0.9288 - val_loss: 0.1706 - val_accuracy: 0.9501
Epoch 49/50 - loss: 0.2306 - accuracy: 0.9280 - val_loss: 0.1701 - val_accuracy: 0.9509
Epoch 50/50 - loss: 0.2222 - accuracy: 0.9333 - val_loss: 0.1698 - val_accuracy: 0.9506
PyTorch baseline - test_loss: 0.1752 - test_accuracy: 0.9508

Running experiment: Wider + deeper
Training on device: cuda
Epoch 01/25 - loss: 0.5372 - accuracy: 0.8520 - val_loss: 0.2340 - val_accuracy: 0.9316
Epoch 02/25 - loss: 0.2110 - accuracy: 0.9385 - val_loss: 0.1659 - val_accuracy: 0.9507
Epoch 03/25 - loss: 0.1516 - accuracy: 0.9557 - val_loss: 0.1392 - val_accuracy: 0.9585
Epoch 04/25 - loss: 0.1177 - accuracy: 0.9641 - val_loss: 0.1124 - val_accuracy: 0.9665
Epoch 05/25 - loss: 0.0957 - accuracy: 0.9708 - val_loss: 0.0974 - val_accuracy: 0.9716
Epoch 06/25 - loss: 0.0805 - accuracy: 0.9750 - val_loss: 0.0957 - val_accuracy: 0.9716
Epoch 07/25 - loss: 0.0678 - accuracy: 0.9797 - val_loss: 0.0879 - val_accuracy: 0.9738
Epoch 08/25 - loss: 0.0582 - accuracy: 0.9818 - val_loss: 0.0858 - val_accuracy: 0.9742
Epoch 09/25 - loss: 0.0487 - accuracy: 0.9848 - val_loss: 0.0845 - val_accuracy: 0.9758
Epoch 10/25 - loss: 0.0444 - accuracy: 0.9858 - val_loss: 0.0849 - val_accuracy: 0.9758
Epoch 11/25 - loss: 0.0395 - accuracy: 0.9876 - val_loss: 0.0779 - val_accuracy: 0.9770
Epoch 12/25 - loss: 0.0363 - accuracy: 0.9877 - val_loss: 0.0778 - val_accuracy: 0.9776
Epoch 13/25 - loss: 0.0316 - accuracy: 0.9894 - val_loss: 0.0797 - val_accuracy: 0.9772
Epoch 14/25 - loss: 0.0295 - accuracy: 0.9905 - val_loss: 0.0775 - val_accuracy: 0.9783
Epoch 15/25 - loss: 0.0270 - accuracy: 0.9912 - val_loss: 0.0771 - val_accuracy: 0.9793
Epoch 16/25 - loss: 0.0231 - accuracy: 0.9924 - val_loss: 0.0780 - val_accuracy: 0.9791
Epoch 17/25 - loss: 0.0213 - accuracy: 0.9928 - val_loss: 0.0789 - val_accuracy: 0.9796
Epoch 18/25 - loss: 0.0207 - accuracy: 0.9931 - val_loss: 0.0829 - val_accuracy: 0.9782
Epoch 19/25 - loss: 0.0184 - accuracy: 0.9938 - val_loss: 0.0887 - val_accuracy: 0.9776
Epoch 20/25 - loss: 0.0200 - accuracy: 0.9936 - val_loss: 0.0876 - val_accuracy: 0.9776
Epoch 21/25 - loss: 0.0181 - accuracy: 0.9937 - val_loss: 0.0829 - val_accuracy: 0.9788
Epoch 22/25 - loss: 0.0182 - accuracy: 0.9939 - val_loss: 0.0822 - val_accuracy: 0.9796
Epoch 23/25 - loss: 0.0161 - accuracy: 0.9944 - val_loss: 0.0875 - val_accuracy: 0.9799
Epoch 24/25 - loss: 0.0156 - accuracy: 0.9946 - val_loss: 0.0801 - val_accuracy: 0.9810
Epoch 25/25 - loss: 0.0148 - accuracy: 0.9950 - val_loss: 0.0846 - val_accuracy: 0.9809
Wider + deeper - test_loss: 0.0821 - test_accuracy: 0.9805

Running experiment: Deeper network
Training on device: cuda
Epoch 01/25 - loss: 0.7371 - accuracy: 0.7715 - val_loss: 0.2936 - val_accuracy: 0.9147
Epoch 02/25 - loss: 0.2767 - accuracy: 0.9189 - val_loss: 0.1948 - val_accuracy: 0.9393
Epoch 03/25 - loss: 0.1938 - accuracy: 0.9432 - val_loss: 0.1497 - val_accuracy: 0.9545
Epoch 04/25 - loss: 0.1536 - accuracy: 0.9549 - val_loss: 0.1283 - val_accuracy: 0.9603
Epoch 05/25 - loss: 0.1232 - accuracy: 0.9644 - val_loss: 0.1096 - val_accuracy: 0.9664
Epoch 06/25 - loss: 0.1039 - accuracy: 0.9697 - val_loss: 0.1037 - val_accuracy: 0.9691
Epoch 07/25 - loss: 0.0897 - accuracy: 0.9735 - val_loss: 0.0979 - val_accuracy: 0.9695
Epoch 08/25 - loss: 0.0759 - accuracy: 0.9770 - val_loss: 0.0969 - val_accuracy: 0.9710
Epoch 09/25 - loss: 0.0696 - accuracy: 0.9776 - val_loss: 0.0874 - val_accuracy: 0.9742
Epoch 10/25 - loss: 0.0619 - accuracy: 0.9813 - val_loss: 0.0848 - val_accuracy: 0.9754
Epoch 11/25 - loss: 0.0526 - accuracy: 0.9840 - val_loss: 0.0828 - val_accuracy: 0.9763
Epoch 12/25 - loss: 0.0479 - accuracy: 0.9847 - val_loss: 0.0862 - val_accuracy: 0.9768
Epoch 13/25 - loss: 0.0429 - accuracy: 0.9864 - val_loss: 0.0948 - val_accuracy: 0.9742
Epoch 14/25 - loss: 0.0409 - accuracy: 0.9872 - val_loss: 0.0854 - val_accuracy: 0.9775
Epoch 15/25 - loss: 0.0337 - accuracy: 0.9889 - val_loss: 0.0844 - val_accuracy: 0.9778
Epoch 16/25 - loss: 0.0321 - accuracy: 0.9898 - val_loss: 0.0870 - val_accuracy: 0.9783
Epoch 17/25 - loss: 0.0326 - accuracy: 0.9897 - val_loss: 0.0848 - val_accuracy: 0.9782
Epoch 18/25 - loss: 0.0299 - accuracy: 0.9903 - val_loss: 0.0860 - val_accuracy: 0.9786
Epoch 19/25 - loss: 0.0283 - accuracy: 0.9909 - val_loss: 0.0818 - val_accuracy: 0.9786
Epoch 20/25 - loss: 0.0250 - accuracy: 0.9920 - val_loss: 0.0870 - val_accuracy: 0.9788
Epoch 21/25 - loss: 0.0234 - accuracy: 0.9928 - val_loss: 0.0824 - val_accuracy: 0.9798
Epoch 22/25 - loss: 0.0236 - accuracy: 0.9926 - val_loss: 0.0840 - val_accuracy: 0.9798
Epoch 23/25 - loss: 0.0210 - accuracy: 0.9932 - val_loss: 0.0958 - val_accuracy: 0.9791
Epoch 24/25 - loss: 0.0201 - accuracy: 0.9935 - val_loss: 0.0877 - val_accuracy: 0.9798
Epoch 25/25 - loss: 0.0208 - accuracy: 0.9935 - val_loss: 0.0871 - val_accuracy: 0.9802
Deeper network - test_loss: 0.0855 - test_accuracy: 0.9803

Best experiment: Wider + deeper with test_accuracy = 0.9805
```




```
Optimization summary:
- PyTorch baseline: layers=(32,), dropout=0.3, lr=0.003, batch=4000 -> test acc=0.9508
- Wider + deeper: layers=(256, 128), dropout=0.2, lr=0.001, batch=256 -> test acc=0.9805
- Deeper network: layers=(256, 128, 64), dropout=0.2, lr=0.0008, batch=256 -> test acc=0.9803

Which changes improved the model:
- Increasing hidden layer size and adding a second hidden layer improved representational capacity.
- A smaller batch size and a lower learning rate improved validation stability and final accuracy.
- Moderate dropout (0.2) helped generalization without overly slowing convergence.

Which changes did not help:
- The smallest baseline network with only 32 hidden units was too limited for top MNIST accuracy.
- Very high dropout or too aggressive regularization would reduce peak performance.
```

Exercise 6: Create a summary table

At the end of the notebook, prepare a **summary table** that compares your major experiments. Include at least the following rows:

- NumPy baseline model
- PyTorch baseline model

- Best optimized PyTorch model

Your table should include at least these columns:

- Implementation (`NumPy` or `PyTorch`)
- Model configuration
- Epochs
- Batch size
- Learning rate
- Final training accuracy
- Final validation accuracy
- Test accuracy
- Final training loss
- Final validation loss
- Key observations

After the table, write a short reflection discussing what you learned from implementing neural networks manually in NumPy versus using a framework.

```
# Exercise 6 - Summary table
summary_columns = [
    "Implementation",
    "Model configuration",
    "Epochs",
    "Batch size",
    "Learning rate",
    "Final training accuracy",
    "Final validation accuracy",
    "Test accuracy",
    "Final training loss",
    "Final validation loss",
    "Key observations",
]

# Helper to safely extract metrics from a history dictionary.
def get_metric(history, name):
    if history is None:
        return np.nan
    return history.get(name, [np.nan])[-1] if isinstance(history.get(name, None), list) else np.nan

summary_rows = [
    {
        "Implementation": "NumPy",
        "Model configuration": "1 hidden layer, 32 units, ReLU",
        "Epochs": 50,
        "Batch size": 4000,
        "Learning rate": 0.003,
        "Final training accuracy": get_metric(globals().get("numpy_history"), "accuracy"),
        "Final validation accuracy": get_metric(globals().get("numpy_history"), "val_accuracy"),
        "Test accuracy": globals().get("numpy_test_accuracy", np.nan),
        "Final training loss": get_metric(globals().get("numpy_history"), "loss"),
        "Final validation loss": get_metric(globals().get("numpy_history"), "val_loss"),
        "Key observations": "Manual NumPy implementation shows core concepts clearly but is slower and harder to tune.",
    },
    {
        "Implementation": "PyTorch baseline",
        "Model configuration": "1 hidden layer, 32 units, dropout=0.3",
        "Epochs": 50,
        "Batch size": 4000,
        "Learning rate": 0.003,
        "Final training accuracy": get_metric(globals().get("results", [None])[0].get("history") if globals().get("results") else None, "accuracy"),
        "Final validation accuracy": get_metric(globals().get("results", [None])[0].get("history") if globals().get("results") else None, "val_accuracy"),
        "Test accuracy": globals().get("results", [None])[0].get("test_accuracy") if globals().get("results") else np.nan,
        "Final training loss": get_metric(globals().get("results", [None])[0].get("history") if globals().get("results") else None, "loss"),
        "Final validation loss": get_metric(globals().get("results", [None])[0].get("history") if globals().get("results") else None, "val_loss"),
        "Key observations": "Baseline PyTorch is easy to train but the small architecture may not reach 96%.",
    },
    {
        "Implementation": "PyTorch optimized",
        "Model configuration": "2 hidden layers, 256 + 128 units, dropout=0.2",
        "Epochs": 25,
        "Batch size": 256,
        "Learning rate": 0.001,
        "Final training accuracy": get_metric(globals().get("best_result", {}).get("history"), "accuracy"),
        "Final validation accuracy": get_metric(globals().get("best_result", {}).get("history"), "val_accuracy"),
        "Test accuracy": globals().get("best_result", {}).get("test_accuracy", np.nan),
        "Final training loss": get_metric(globals().get("best_result", {}).get("history"), "loss"),
        "Final validation loss": get_metric(globals().get("best_result", {}).get("history"), "val_loss"),
        "Key observations": "Larger model and tuned optimizer settings produced the best test accuracy.",
    },
]

summary_table = pd.DataFrame(summary_rows, columns=summary_columns)
summary_table
```

	Implementation	Model configuration	Epochs	Batch size	Learning rate	Final training accuracy	Final validation accuracy	Test accuracy	Final training loss	Final validation loss	Key observations
0	NumPy	1 hidden layer, 32 units, ReLU	50	4000	0.003	NaN	NaN	NaN	NaN	NaN	Manual NumPy implementation shows core concept...
1	PyTorch baseline	1 hidden layer, 32 units, dropout=0.3	50	4000	0.003	0.933250	0.950583	0.9508	0.222171	0.169838	Baseline PyTorch is easy to train but the smal...
2	PyTorch optimized	2 hidden layers, 256 + 128 units, dropout=0.2	25	256	0.001	0.995042	0.980917	0.9805	0.014823	0.084593	Larger model and tuned optimizer settings prod...

Next steps: Generate code with `summary_table` New interactive sheet